



# **A Culture of Uncertainty: Interaction and Organizational Memory in Software Engineering Teams under a Productivity Scheme**

Organization Studies

1–20

© The Author(s) 2017

Reprints and permissions:

sagepub.co.uk/journalsPermissions.nav

DOI: 10.1177/0170840616685357

www.egosnet.org/os



**Nahoko Kameo**

New York University, USA

## **Abstract**

Drawing on an ethnography of software engineering teams that introduced a new and empowerment-centered productivity scheme, this study delineates how, despite management's affirmation of its commitment, software engineers produced and reproduced a 'culture of uncertainty' characterized by constant doubt about how long the scheme would last. Engineers shared previous experiences of failed productivity schemes and collectively used this organizational memory to understand their new situation. Workers drew on this organizational memory in everyday interaction to sustain a culture in which everyday management decisions served as indicators of management's potential abandonment of the scheme; as a result, workers remained uncommitted to the scheme. Workers interactionally employed organizational memory as a resource that they used to interpret and respond to changes. Analysis of this process shows the links between organizational memory, culture creation, and culture's influence on productivity scheme changes.

## **Keywords**

ethnography, resistance to change, organizational memory, culture creation, empowerment, scrum

## **Introduction**

Drawing from a year of field observation at a software engineering organization that employed a new productivity scheme, this article delineates how the workers' culture refracted the way the new scheme shaped their practices and experiences. The article builds a theoretical model of the influence of organizational memory, culture, and interaction on worker resistance. The model explains how culturally embedded memories of previous productivity-scheme changes led

---

### **Corresponding author:**

Nahoko Kameo, Department of Sociology, New York University, 295 Lafayette Street, 4th Floor, New York, NY 10012-9605, USA.

Email: [nahoko.kameo@nyu.edu](mailto:nahoko.kameo@nyu.edu)

engineers to resist a productivity scheme that had been designed to empower them and increase their autonomy.

Empirically, the study contributes to our understanding of barriers to productivity schemes in general and empowerment schemes in particular. As I show, workers interpreted the new productivity scheme through the lens of organizational memory. The legacy of failed productivity schemes loomed large: workers remembered how other schemes had come and gone and understood the new scheme as another one of ‘those’ schemes that could be cancelled at any point by managers’ change of heart. Workers constantly talked to one another and collectively interpreted managers’ actions as potential signals of the productivity scheme’s demise. Thus, the study shows the importance of attending to the intertwining of organizational memory and creation of group culture in interaction when an organization employs a new productivity scheme.

The article begins by evaluating existing research on culture in organizations. Previous studies on resistance to change have identified organizational culture as the barrier to change practices. However, these studies have not adequately explained the durability of cultural schemas in organizations. I address this lack by developing an approach to workers’ response to workplace change that attends to how workers create and maintain an organizational culture: they collectively draw on organizational memory to make sense of their current situations in interactions. Workers drew heavily on organizational memory of previous productivity schemes’ failures as they sought to interpret managers’ actions. Using insights from Giddens’ structuration theory and Chicago traditions of work ethnography, the article builds a theoretical model of worker resistance that incorporates organizational memory, culture, and interaction. It explains how a productivity scheme designed to empower workers and increase their autonomy resulted in a culture that undermined the principles of empowerment and the scheme itself.

### *Memory and interaction in organizations*

Over the past couple of decades, inquiry into the role of culture in organizations and organizational change has blossomed (see, e.g., Fayard & Van Maanen, 2015; Schein, 1985; Pfeffer, 1981; Sheridan, 1992; Weeks, 2003). People in organizations create and share distinctive sets of durable schemes, stories, rituals, and routines that guide their actions (Fine, 1984; Fine & Hallet, 2014; Morrill, 2008; Schein, 1985). While culture has sometimes been hailed as an important determinant of productivity (Ouchi, 1981; Pfeffer, 1981), its durability also makes it the culprit of resistance to change, whether such change is due to environmental shifts (Hannan & Freeman, 1984) or managerial decisions (Beer, Eisenstat, & Spector, 1990; Kanter, 1989; Kanter et al., 1992; Kirkman & Shapiro, 1997; Schaffer & Thompson, 1992). Because organizational culture shapes how workers interpret and respond to new situations, workers under conditions of change may voice objections, not follow new rules, or covertly undermine changes that are not compatible with their taken-for-granted assumptions about the organization (Labianca et al., 2000; Zucker, 1977).

This renewed interest in organizational culture (see Morrill, 2008) needs careful treatment. If we attribute failure to change to cultural resistance, we run the risk of portraying workers as ‘the’ cause of failure: change fails because workers are unable to adapt to it despite organizational needs. Worker resistance then appears fundamentally ‘irrational and dysfunctional’, even though the resistance makes perfect sense to the workers (Ford et al., 2008). Instead, resistance to change is a product of organizational members’ actions and interactions in the organization, and ‘situated discursive and nondiscursive practices that are simultaneously enabling and constraining, coherent and contradictory, complex and simple, efficacious and ineffectual’ (Mumby, 2005, p. 20). To accurately understand how culture relates to worker resistance, then, we must stop treating culture as a fairly stable scheme that was created in the past and is therefore independent of current

interactions. Instead, we need to look at the co-constitutive relationship between the ‘past’ that organizational members share and their current practices. To understand how culture influences workers’ actions, then, we need to theorize the link between organizational memory – the historical construct that members draw on – and workers’ actions.

Organizational memory is not a static and objective portrait of the organization’s history; it is a selective set of reconstructions of the past (Walsh & Ungson, 1991; Rowlinson et al., 2009). Organizational memory is the shared recognition of collective experience, and it is reenacted every time workers interpret their current situation. Workers use this reenacted organizational memory as a guidepost in making sensible and appropriate choices about the present and future (Walsh & Ungson, 1991; Olick, 2003; Olick & Robbins, 1998). As Halbwachs notes (1980), the past is always relevant to the degree that it relates to people’s present concerns and thus is always subject to reinterpretation. These reinterpretations are constrained by previous renderings (Olick, 2007), but organizational memory is flexible enough to serve as a resource that organization members can draw upon when facing practical problems (Swidler, 1986).

Several scholars over the past two decades have discussed the importance of memory in organizational studies (Walsh & Ungson, 1991; Moorman & Miner, 1998; Fiedler & Welp, 2010; Wadhvani & Bucheli, 2014). However, there is a dearth of studies that explore the relationships between organizational memory, culture creation, and reactions to change *in situ*. This article responds to the need for an empirically grounded model that explains how organization members take up organizational memory, build and sustain meaning systems, use these cultural schemas to interpret situations, and then draw on these interpretations to respond to organizational change. The article goes beyond simply equating organizational memory with cognitive frameworks; instead, it looks at organizational actors in interaction to see how organizational memory becomes a salient prism through which workers understand their current situation in the organization.

My choice of ethnography as a methodological approach was also driven by this theoretical question. Cultural schemas shape organization members’ actions, which then recursively (re)create the schema. If we want to study such processes of structuration and understand how members use organizational memory to constitute cultural schema, it is essential to conduct continuous observation of workplace interactions (Barley & Tolbert, 1997). As scholars of memory work note, remembering the past is a constitutive part of understanding the present (Olick, 2007, Olick & Robbins, 1998) and using projective capacity to decide on courses of action (Mische & Emirbayer, 1998). It is in these moments of interaction that people ‘organize’ organizations (Barley & Tolbert, 1997; Bechky, 2011; Hallet, 2010; Hallet & Ventresca, 2006). By zeroing in on such moments, this article advances our understanding of workers’ agency and responses to change without blackboxing such responses as ‘resistance’. Interactionist theory guides researchers to look at the situations through which organizational members interpret and then respond to a productivity scheme change and the effects of these responses on the organization’s culture.

### ***A new software engineering productivity scheme: Scrum***

In the case I studied, a software engineering organization introduced a new productivity scheme called ‘Scrum’. Scrum is one of the ‘agile’ methods that gained popularity in the mid-1990s. ‘Agile’ methods emerged as alternatives to what creators termed the ‘traditional’ Waterfall model of software engineering. In the Waterfall model, the whole software engineering project is divided into large phases of design, development, and testing, and each phase must be completed before the project can proceed to the next phase. This model, though still commonly used in the industry, has been criticized as rigid and risky because it cannot incorporate changes that occur mid-project (Royce, 1970). Agile methods arose to overcome the Waterfall model’s perceived problems.

Scrum is a management technique consisting of a set of rules designed to empower engineering teams and a set of rules about software engineering. Like other agile methods, Scrum features shorter development cycles with smaller releases of added new features or new versions, as well as increased communication and collaboration over managers' direction and documentation (Larman, 2004). Scrum has gained popularity in the industry, and many large organizations (such as IBM) have adopted it with the help of consultants.

From an organizational perspective, the biggest changes required for successful Scrum implementation are shifts in organizational structure and decision-making practices. Scrum requires a flatter organization, devoid of the middle-range managers who typically direct software-production teams. Instead, Scrum dictates that software-production teams be self-managed. Indeed, this is seen as the essential but most difficult part of Scrum to fully implement. One of the founders of Scrum, Schwaber (2004), describes cases in which Scrum implementation failed and then states: 'Those examples show how easy it is for people to misunderstand Scrum ... They don't understand that Scrum involves a paradigm shift from control to empowerment, from contracts to collaboration, and from documentation to code' (p. 26).

According to Scrum literature, the scheme is meant to organize software production as a collaboration between stakeholders and to minimize the need for management control for production. 'Scrum Masters' are chosen from among engineers and serve as the equivalent of project managers (Schwaber, 2004, p. 16), but they are meant to be facilitators, not managers, who serve only to help the team make decisions and mediate between upper management and the Scrum Team (Larman, 2004, p. 115.) The people who are engaged in actual production – including Product Owners (who represent customers and request what they need for the software), the Scrum Master, and the Team – have strong control over the output and schedule of work. All others, including managers, have to work through the team and have 'no direct control over the project's execution or progress' (Schwaber, 2004, p. 15). In short, Scrum dictates that 'although it might seem counterintuitive that Teams are managers, one of the central tenets of Scrum is that Teams handle their own management' (p. 15).

Literature on the implementation of Scrum is largely found in Management Information Systems (MIS) research. This literature points to a large and stable trend in software production that recommends increasing software engineers' self-management and minimizing management overhead. Thus, the challenge is seen as that of reorienting software engineers from individual work to self-managed teamwork. MIS researchers seem to agree that social factors are key to successful implementation. Moe et al. (2010), for example, note that 'trust and shared mental models' were important for a successful implementation of Scrum. Studies (e.g. Abrahamsson et al., 2003; Boehm & Turner, 2005; Nerur et al., 2005) indicate that most firms that decide to implement Scrum (or similar agile methods) have been operating under traditional, more hierarchical models of software production and that the transition is often blocked by the very ways that these previous work practices have affected the structure and culture of the organization. For instance, managers may continue to try to control the way software engineers plan their work, despite Scrum's vision of empowerment, or they may fail to allocate the resources and time necessary for the team to self-manage its production (Cohn, 2010; Moe et al., 2010; Boehm & Turner, 2005).

Although MIS researchers seem to agree on the obstacles to implementing Scrum (Nerur et al., 2005), they are less clear on the actions to take to cultivate what Moe et al. termed 'trust and shared mental models'. Abrahamsson et al. (2003) thus call for more empirical research that takes a deeper look at what goes on in the process of transition from traditional work models to Scrum. This article responds to that call by analyzing a case in which software engineers were unable to transition to the new interpretative scheme.

## Data and Methods

### Settings

CIS is a well-known US-based international manufacturer of office-automation equipment (all company and individual names are pseudonyms used to ensure confidentiality). Its software-engineering division is made up of several business units, each of which includes many smaller teams. The teams are responsible for developing and maintaining software products that control the office-automation equipment that the company sells. The software engineering division consists of approximately 16 business units distributed across the country and abroad, but its two main offices are located in an East Coast city and in the large, post-industrial, West Coast city where I conducted my study.

The company enjoyed an almost complete monopoly on the market between the 1950s and the 1970s, but it now faces fierce competition from other companies in the United States and Northeast Asia. Office-automation equipment is now easily mass-produced, and its price has gone down sharply. Consequently, the software-engineering division has been under cost pressures that began in the 1980s but have intensified in recent years and have led to frequent layoffs of software engineers. Over the last 25 years, following the cancellation of large software projects and the emergence of offshore development, the organization laid off approximately two-thirds of its West Coast employees because these offices now have no managerial functions and only deal with production. The layoffs coincided with previous change efforts, as the firm tried to restructure and improve the software-engineering division at the same time. Remaining engineers were reassigned to form new teams in a new and completely restructured organization.

The quality of the software and the timeliness of new releases have also been sources of contention within the company. Because of the restructuring, layoffs, and reorganization, some software-development teams found it hard to maintain production quality and meet demands for cost and timely delivery. In one business unit, the executive decided to implement Scrum in hopes of increasing productivity and cutting costs without further layoffs. During the period of my field observation between 2007 and 2008, all the teams on the East Coast and three of the five teams on the West Coast were practicing Scrum. The two teams that did not adopt Scrum were excused because they did maintenance work rather than software development and their members' seniority and expertise were stable.

I conducted field observations with two teams practicing Scrum within one business unit in the software engineering division. That business unit was responsible for manufacturing and maintaining all the software for a series of products. The business unit had ten teams: five on the East Coast and five on the West Coast. The five teams on the East Coast were responsible for most managerial functions and software development, while the five teams on the West Coast mainly developed particular functions of the software and maintained the software for older products. Each team consisted of approximately eight to 15 engineers, and in total, the organization employed about 70 software engineers in its West Coast offices.

The two teams I observed consisted of approximately 25 engineers, all of whom worked under one line manager. Their tenure in CIS varied, but the majority of them had been there for more than ten years; each team had a few engineers who had been hired recently and had worked for CIS for less than five years. In this article I distinguish four levels of hierarchy. I call the top manager of the software division the 'executive'. I call the next level of managers, who were in charge of a particular subsystem and were responsible for about 60 engineers each 'second-level managers'. 'Line managers' were directly responsible for one or two teams of engineers. 'Software engineers', more commonly called 'engineers', filled the largest part of the hierarchy. Since the corporation was large and had many product lines, it also included three

to four layers of management above the 'executive' and up to the CEO, but since they are not directly relevant to my analysis here, they are not described.

My fieldwork started several months after the introduction of Scrum. The organization gradually started to introduce the new process beginning early in the summer of 2007, first experimenting by applying it to one small team. I made preliminary visits to the field site beginning in August 2007. I conducted fieldwork from January through December 2008, usually visiting the workplace two or three times a week for a full workday (eight to ten hours). Additionally, I conducted 28 in-depth interviews and numerous short, one-on-one casual conversations with engineers and managers. These interviews enabled me to discern what kinds of experiences and expectations software engineers and managers had before Scrum implementation and to track their changing opinions about Scrum.

While in the field, I observed how engineers and managers worked. I observed software engineering work, design meetings, managerial meetings, corridor communications, and conversations in informal settings. I had permission to study these processes with the management's consent, and in return I was expected to share observations and insights into the changes that Scrum produced in the organization. The majority of the organization's members were cooperative: they allowed me to observe their work, answered the many questions I asked, and sat for numerous informal interviews. I conducted one- to two-hour interviews with some of these managers and engineers to gain more insight into the organization and their experiences of Scrum's 'empowerment' paradigm. Although I did participant observations and some in-depth interviews with managers, the vast majority of data I collected concerned engineers and their interactions. Thus, my analysis primarily focuses on engineers and their culture creation. I used the interviews with managers to understand the official accounts of organizational change and general perceptions regarding past productivity schemes.

I introduced myself to software engineers as a social scientist who was interested in organizational culture and was trying to understand the social aspect of software engineering. I was also a graduate student at the time, and the software engineers quickly began to refer to me as such – a graduate student trying to get a degree by conducting a thesis project. I thus gained the position of a harmless observer with whom workers could share their opinions and feelings about their work lives. Workers confided in me about their worries about Scrum and often told me about their career contingency plans (as well as worrying about my plans for after graduate school).

I was allowed to carry a notebook wherever I went, and I usually jotted down as many details as possible about conversations and interactions that I observed. I wrote field notes based on these jottings as soon as I left the field for the day. I was also allowed to audio-record formal meetings; thus, I audio-recorded and then transcribed the managerial and team meetings I attended, as well as all one-on-one interviews. I ended up with more than a thousand pages of interview transcripts and field notes, and I also drew on archival materials, including meeting minutes, email messages, company handbooks, and organizational charts.

I analyzed my field notes and documents using a grounded theory approach, closely following Emerson's approach to inductive theorizing (see Emerson et al., 1995; Glaser & Strauss, 1967). Thus, I repeatedly coded field notes and interviews. In the first few rounds, I was engaged in open coding – themes included 'choosing which task to work on', 'working in a common room', 'manager-engineer interaction', 'complaints about the team', 'who to ask for help', and so on. As my coding progressed, I was able to reduce these initial themes to three analytic themes: 'issues of hierarchy', 'memories of past productivity schemes', and 'talking about Scrum'. Subsequently, I chose the analytic theme of engineers' group culture, because I realized that organizational memory had to be instantiated by engineers, since it was in interviews and conversations with engineers that this theme arose. Similarly, 'issues of hierarchy' were a problem precisely because engineers

and managers found it so. Interaction among engineers was the key to understanding how the three themes connected and how Scrum was reshaping the organization. Once I decided on the project's analytic theme, I reread all my field notes and interview transcripts to conduct more focused coding and then wrote numerous analytic memos to further develop the themes of interactional creation of engineers' group culture and that culture's relationship with organizational memory. I also continuously read theories of interactionism, ethnographies of work, and research on group culture before, during, and after the fieldwork period. This article is a product of an iterative process between data gathering, data analysis, and theorization (see Katz, 2001; Tavory & Timmermans, 2014).

## Findings

### *The initial change: Background and summary*

With the implementation of Scrum, old software development teams were disbanded and reformed as two 'Scrum teams'. Engineers divided themselves into two teams so that their skills would be equally distributed without managers' involvement. Engineers on each team were told to work as a team and to be self-managed and self-organized. They were provided a common area (called the Scrum room) as well as private offices, and they were strongly encouraged to work in the common area so that it would be easier to collaborate. A 'burn-down chart' on the wall of the common area depicted each team's progress by displaying the number of hours the team expected to spend on each task. Each engineer was supposed to indicate their status on their task so all members would know what the others were working on.

Teams also had the authority to decide which and how much work to schedule within each two-week development period. They were to make these decisions based on a long list of prioritized tasks provided by 'Proxies of Product Owner' – technical experts who represented the company's customers. After the team had decided what their tasks would be, they discussed and decided together both how much work each task would involve and who in the team would work on which tasks. Each team assigned one engineer to become a 'Scrum Master' who facilitated the team's decision-making and worked to remove any obstacles (called 'remove barriers' in Scrum terminology) to meeting the delivery schedule (such as negotiating when to use testing machines, for example).

As teams were given more autonomy and decision-making authority, line managers' roles diminished, as recommended by Scrum. Line managers were not allowed to direct engineers regarding the execution of software development and were only allowed to help them 'remove barriers' when explicitly asked to do so by engineers. They had two remaining roles: coaching interpersonal skills and evaluating the accomplishments of individuals.

Six months after Scrum was implemented, there was a reorganization. A new line manager, Josh, was brought in from another division of the company to manage the two teams. Additionally, a new second-level manager, Luke, moved up from managing two smaller teams that were not practicing Scrum to overseeing all West Coast teams, including the Scrum teams. Both managers and engineers knew Luke to have a 'great track record', but engineers did not know his attitude toward Scrum; all teams he had managed previously were not doing Scrum.

I begin my analysis in the next section by showing that the managers and engineers alike shared a memory of the organization as one that was always trying a new scheme to increase productivity, only to abandon it after a few years. This organizational memory, however, existed in the intertwining of remembering the past, understanding the present, and devising new actions. Therefore, in the subsequent section, I show how workers employed and constituted this organizational memory in interaction: together, engineers interpreted management decisions through the lens of the

organizational memory and, by doing so, sustained collective uncertainty toward Scrum. I trace the situations of talk at work in which engineers collectively scrutinized management actions as potential signals of Scrum abandonment, cast doubt on the legitimacy of Scrum, and presented a lack of commitment to the productivity scheme.

### *Organizational memory among workers*

One of the challenges of workplace change is rewriting the organization's past (Anteby & Molnar, 2016). The introduction of new procedures inevitably requires new ways of making meaning, and new meanings must somehow be amenable to the organization's narrative about itself. In CIS's case, two elements of the past seemed to strongly influence how engineers interpreted the introduction of Scrum. First, the past authority-and-control system, in which each line manager directed 'his' team, provided a familiar prototype for interaction despite Scrum's empowerment paradigm (see also Labianca et al., 2000). Second, engineers and managers alike remembered that CIS had implemented quite a few productivity schemes in the past decade, only to abandon them after a few years. Since the 1990s, CIS had tried at least three management techniques: CMM (Capability Maturity Model; see Paulk et al., 1995), Critical Chain (Leach, 2000), and Feature-Driven Development (see Palmer & Felsing, 2002). In the office, engineers and managers commonly referred to this organizational history of trying out different management 'processes' only to move onto the next one. Although Scrum was substantively different from the other processes that the organization had tried, what organizational members remembered was not the content of the schemes themselves, but rather that those previous 'processes' had come and gone.

Engineers and managers often mentioned CMM to me while we were discussing Scrum. CMM is a productivity scheme that involves documenting every process so that the best practices can be replicated and CIS had implemented this scheme in the late 1990s. At the time, CMM had been mandated throughout the organization. A task force had been created to apply CMM and ensure that the organization was properly certified. Yet, it had lasted for only three years – a short time relative to many engineers' tenure – after which it had been replaced by a completely different productivity scheme. It had also resulted in some human resources turbulence. A veteran engineer and a second-level manager both described their bitter experience with CMM in separate interviews. A veteran engineer, who had worked for CIS for more than 30 years, shared in an interview how he felt about Scrum and the organization:

*Roger:* [Talking about Scrum] We constantly are tweaking processes if we find something falling short, or if somebody has an inch, they want something else measured, or they want to solve some other problem, they'll tweak the process. Processes are constantly changing. I guess that's to be expected. But I find that engineers are often left behind when processes change.

*Interviewer:* What do you mean, left behind?

*Roger:* Meaning that their needs and their requirements are not always accounted for. [The interview continues. After detailing different projects he has worked for, the topic of 'transformations the organization has gone through' comes up. We talk about Feature-Driven Development a bit, and then I ask about CMM.]

*Roger:* We went through that too. When we went through our level-two assessments, I thought it was one of the biggest jokes I had ever seen.

*Interviewer:* Okay. Because?

*Roger:* Because we filled boxes of paper, we produced all this documentation saying we had done this process and that process and that process, show it to the

- assessors, literally, paper boxes full of documents. So they went in, they saw all these boxes of paper, and they said, cool, you're level two.
- Interviewer:* Who are 'they'?
- Roger:* Whoever the assessors were – somebody from Carnegie Mellon, I suppose. We had internal assessors who sort of set us up for the real assessors. And then the real assessors came in and they kind of walked through for a day, and they were gone, and the next thing we knew we were level two.
- Interviewer:* Which was important.
- Roger:* It was important to somebody.
- Interviewer:* How long did it last?
- Roger:* Maybe it lasted two years, the late 1990s. And it just faded away. Then, a little while later, they sort of came up with the idea of Feature Driven Development ...

In this excerpt, the engineer describes the generalized 'process' as having been forced, having been of no help to engineers, and having died out within a few years. He then moves onto explaining the next 'process' he experienced in the organization: Feature-Driven Development. Although the two productivity schemes have little content in common, in the memory of the engineer, they formed a series of management fashions that did not work. In fact, Roger defended Scrum by saying, 'I thought it made more sense than some of the other ones we did'. But his comparison also presumed that Scrum was one of many productivity schemes that he had experienced and would continue to experience at CIS.

The next excerpt – from Luke, a second-level manager – shows how he thinks about 'the processes' and, additionally, that he remembers the introduction of a new scheme as part of the company's restructuring efforts. After discussing the layoffs and big restructuring of engineering units and projects, the line manager described when CIS started looking into 'modern project management or modern development management or so on':

- Interviewer:* What was the rationale to do CMM to start with?
- Luke:* Well, because I think [the head of the organization at that time] was an alumni of Carnegie Mellon. One of the stupid things, then, you know, all the politics, they wanted to please him ... and so on, and they say, 'Well, that's the model'. This is the, 'This sounds good!', 'This sounds good!' – you know what I mean? But evidently we didn't really change our mindset and the basic attitude of people. So basically we went through CMM assessment, CMM 2, and even CMM 3 [...] we passed the assessment but, you know, it was a hundred percent artificial! [...] We didn't solve anything with CMM.  
[Pause]
- Luke:* Then what happened was we found ourselves in big trouble. Now, we needed to fire people. So what happened was the very first layer we fired was all the infrastructure people around CMM. You know, when it comes to fire people [who are] writing code or people talking, guess what? The choice was made, everything that was put in place that was somehow keeping this organization process oriented – all that was just wiped out.  
[Interview continues with the history of the organization and how CIS started to implement different productivity schemes after CMM.]
- Luke:* Then suddenly we got a note, small note to senior staff saying, 'Well, nobody is going to use Circuit [the software that was used for feature-driven management] anymore; we are going to shut it down'. We just had spent a few million

dollars and so on, but that's okay, we failed, no want [sic]. In place of analyzing why we failed ... that was because we buy a toy without really understanding. It's a typical thing. 'I want a toy'. Versus understanding what the problem is. We never analyze our failure. We move to the next thing. We never say what went wrong before; we always say, 'Well, the next thing is going to fix it', but we don't know what's wrong. So now here we say it's Scrum. It's becoming the thing that's going to save us. It'll address all the problems we have, and it's going to do everything for us. That's where we are.

Both Roger and Luke employ organizational memory to make sense of their current situation (Walsh & Ungson, 1991). Productivity schemes, by their account, do not help engineers actually do their work; Roger describes the decision to implement CMM as the result of a manager's whim and connections. As this excerpt exemplifies, engineers and even managers commonly thought that CMM and other management techniques had 'failed' and did not hesitate to speak of them in such terms. Indeed, in a brief interview, Jack, the executive who had decided to try Scrum, told me that the reason he wanted to try Scrum was that the other processes he had tried had never worked.

Organizational memory of 'processes that fail and go' was quite commonly shared, and skepticism toward them was both common and openly held. The resulting resignation that engineers and managers alike felt toward 'processes' was complicated by the fact that these management techniques affected the careers and evaluation of engineers – in the worst case, in the form of layoffs. When the executive who had propelled CMM left, CMM was discarded without much official notice or contestation; as Roger said, 'it faded away'. But in the resulting restructuring, the people who had left software production to implement CMM had been let go. An engineer mused to me during an interview, 'Every Thanksgiving someone has to go, so it became like a tradition!' And, as layoffs occurred, those who were engaged in the last productivity scheme – those who had stopped being engineers to become 'process people' – were targeted for removal because they were deemed less valuable in a critical moment of reorganization. Because the period when CIS was implementing and abandoning CMM coincided with the period when the firm was fiercely restructuring, no one could accurately estimate how many workers had been laid off *because of* their involvement with CMM. The result, however, was that workers' stories of productivity schemes were tied to their stories of layoffs.

When Scrum was implemented, engineers wanted to know both who had decided to implement it and the degree of consistency that they could expect from the management – especially the second-level manager and line manager who directly supervised them – regarding the change. Thus, when there was a reorganization and a new second-level manager was assigned to overview the two Scrum teams I was observing, talk regarding the ramifications of the reorganization on Scrum was rampant throughout the organization. I asked an engineer what he thought of the reorganization. Interestingly, his reply revolved around the question of whether this new manager, Luke, seemed to 'like' or 'not like' Scrum. As the new scheme was considered to be potentially a passing fad, the engineer would not be certain about Scrum's legitimacy unless he knew about Luke's attitude toward it. This excerpt from my field notes begins with my description of Mel knocking on my office door:

After the announcement meeting, I went back to the office to write up a memo. Around 5:30, somebody knocked on the office's door. The door was open, but Mel did it so that I would realize that I had a visitor. We chat a little bit, and then I ask him to explain the reorganization.

'But you were in the meeting!'

'Yes, but I understood only half of it', I said.

‘Okay. So the biggest change for us is that now we have Josh instead of Tom. Tom is going to manage the team Josh was managing.’ We talked about a few other re-assignments of managers, and then the conversation moves to how he thought about Scrum. He said when Scrum started, it was very much Bill’s initiative. ‘Bill was very much into it, and he also wanted a flatter organization, as in the book of Scrum’, he said. ‘But I had mixed feelings about it. Some of it I liked. Other things I didn’t.’ He continues to explain his take on Scrum, and shifts again to the evaluation of changes.

‘The biggest change is Eric’, he continued. ‘For Tom, he just got another team. Josh as well, in his case, two teams each with 10–15 people. But Eric is going back to be an engineer’, he said. I wasn’t sure what to say. Eric was a Scrum Master of another team, but now he was assigned to be one of the engineers in another team. ‘I heard very nice things about Luke.’ He praised his new second-level manager. ‘Luke doesn’t seem to like Scrum so much.’

As the excerpt shows, the engineer took organizational decisions such as promotion and reassignment as indicators of the management’s evaluations and judgments about Scrum’s legitimacy. Because the teams were reassigned to Luke, who had not managed Scrum teams before, Mel decided that his new second-level manager took a certain distance from Scrum. Although all reassignments were given official explanations that had nothing to do with Scrum’s legitimacy, Mel saw them as having immediate ramifications that he needed to interpret. Specifically, two reassignments stood out to the engineers. First, Bill, who had championed Scrum, moved to a supervisory position over the Scrum process but lost the teams he managed to Luke. Second, Eric, who had been the line manager of a team, was reassigned to work as an engineer on another team. Mel treated both changes as demotions that should be interpreted as warning signs regarding Scrum’s slipping legitimacy. Engineers engaged in discussions about the reorganization, and the consensus seemed to be that Scrum’s legitimacy was being contested.

Later the same day, when I returned to my car, I found an engineer called Zian chatting with an engineer from another team:

I put everything in my bag and went to the parking lot. Zian and another engineer were standing just next to my car, chatting. ‘We were talking about the reorg’, Zian said. ‘Oh yeah! It was a big change, wasn’t it?’, I said. They seemed to reach to the conclusion that Scrum will not be practiced so much anymore. ‘Luke doesn’t like Scrum’, Zian said. ‘In a year we’ll not have any Scrum teams’, he pronounced. They continue talking about the human resources changes. They were talking about Eric going back to be an engineer. But in the end Zian shrugged his shoulders and said ‘the salary doesn’t change anyway, so why care about a demotion?’

In this excerpt, two engineers take for granted that some reassignments should be interpreted as ‘demotions’, despite the fact that the reorganization was not officially presented in such terms. The engineers seemed to fill in the gaps they saw in the official explanations of reorganization with their interpretation: that the ‘demotions’ of certain people who were practicing Scrum heralded the fall of the Scrum regime.

As these excerpts suggest, the engineers tied the company’s reorganization to its commitment to Scrum and subjected both to multiple discussions and reinterpretations. For the members of the organization, it went without saying that the reorganizations signaled certain demotions and promotions of people and a possible demotion of the productivity scheme at hand. The organizational memory of past productivity schemes and layoffs shaped engineers’ group perspective (Becker et al., 1977): they understood management techniques to be short term, personnel dependent, unlikely to help them accomplish tasks, and risks to their job security. Thus, rather than accepting Scrum and empowerment unproblematically, engineers saw it as one more ‘process’ that would

soon be abandoned and acted to ensure that the fall of Scrum would not affect their well-being in the organization.

This perspective was not a static cognitive bias resulting from past personal experiences. Rather, it was created in situ as engineers recalled the organizational memory to make sense of the current implementation of Scrum. The perspective further helped create and maintain engineers' culture by continuously assessing whether the management would continue to support Scrum. As I show below, meaning-making in interaction sustained engineers' uncertainty and guided their dispositions toward Scrum. Engineers opted to show that they were engaged with the new productivity scheme only to the extent the management seemed to commit to it. Meanwhile, they kept watch, debating every single management act as a portentous sign of the future of Scrum.

### *A culture of uncertainty: Interactional creation of non-commitment*

While the engineers' perspective arose out of the organizational memory they embodied and their vulnerable organizational position as workers, it was through continuous interaction among the engineers that they produced a collective understanding of the fad-like nature of productivity schemes. The collective understanding then led them to avoid committing to Scrum. Talk at work thus not only enabled coworkers to work together by sharing important work information, but also shaped workers' group identity: through talk, workers created collective meanings, including collective understandings of how to solve problems (Fine, 1984; Fine & Hallett, 2014; Orr, 1996).

The engineers at CIS constantly talked with one another – during meetings, in small talk while working, at the coffee room while heating their lunch, carpooling, and at other work and social occasions. In these conversations, engineers hashed over the smallest of cues to try to guess what management intended to do in the future; together, they interpreted the meanings of management's actions. Thus, even when managers' remarks did not explicitly concern Scrum and empowerment but instead addressed other technical or managerial issues, engineers read them as veiled signals that actually referred to the continuation or discontinuation of Scrum.

At one point, the managers on the East Coast decided to create a team to check the quality of the components of certain software features the company was developing. According to Scrum guidelines, however, it was supposed to be the job of each team's 'Proxies of Product Owner' to define the design requirements and receive the delivered software. The change did not seem to accord with what Scrum books and consultants recommended. My field notes describe how engineers scrutinized this decision:

*Sam:* Does Jack [the executive] know what he is leading to by that?

*Joe:* I think so.

[Then Sam murmurs 'Jack kills Scrum?' and all engineers burst into laughing.]

*Sam:* No, I'm asking, asking. 'Where is the link??'

*Aamit:* So where is ... is this compromise? Of ... Agile or Scrum? What's our feeling? Does it matter? It doesn't matter?

*Joe:* Got it. It was a discussion at the retrospective [meeting] last time too. Have we bought into it? And the reaction was mixed. The general feeling is we haven't bought into 100%, and we probably never will.

*Aamit:* But the point is, is the trend going from 100% to down to 0, or going towards 100%? Who is for and who is not?

[Then, nobody was talking. After an awkward 10 seconds, Mel said, 'Well, it depends on what you mean by Scrum' and others said 'Uh-huh' to move to the next topic.]

The change was significant for engineers because it had been made by the East Coast office – the management headquarters. Although the decision had come from the need to control a software program, the engineers interpreted it as a sign of Jack’s detachment from Scrum because Jack was deviating from what the Scrum manuals said. Since Jack would ultimately be responsible for the final verdict on Scrum for the whole organization, the engineers decided that the scenario ‘Jack kills Scrum’ had just become more likely. They quickly converged on the conclusion that ‘the management may not commit to Scrum’. Sam voiced their shared question when he said, ‘No, I’m asking ... where is the link?’ It was the link between management’s action and their degree of commitment that the engineers were looking for and interpreting together. Engineers paid careful attention to signs from management, interpreted them together, and adjusted their commitment to Scrum accordingly. Their assumption that the management would abandon Scrum was renewed and vindicated every time they interpreted a management action as a potential signal of Scrum abandonment.

Because the engineers understood Scrum as something that the company was currently practicing and could abandon at any time, rather than a set of routines that they could follow uncontested, the question became how much they should follow Scrum practices. When the Scrum Master for the networking team was absent for training, the team members discussed dropping some Scrum practices and ‘read into’ a manager’s everyday remarks as part of making their decision. Mel, a mid-career engineer in his 40s, was serving as substitute Scrum Master for the week:

In the daily Scrum progress meeting, Zian and other engineers were gathering at the Scrum room. Mel stands up, and says ‘let’s Scrum.’ It started from Martin, making clockwise circle. In Zian’s report, he asked if they have retrospective [retrospective meeting, where teams share knowledge and solutions; essential in Scrum practices] today or not. They had not decided when due to some schedule conflicts. Mel said he will talk about it in his turn. After reporting what he did, he brought up that topic.

- Mel:* So, are we having retrospective this afternoon?
- Zian:* Support and Services team stopped doing it. Shall we?  
[Other engineers looked taken by surprise that the other Scrum team skipped it. Joe, a senior engineer, said to the ethnographer, ‘Hey, you witnessed we stop doing Scrum properly?’ I said ‘I’m here!’ and let them continue.]
- Zian:* They just ask if something is important and should be discussed among the team, and if there is none, they don’t have the meeting.
- Aamit:* Yeah. Actually, in one of our meetings once I was sitting next to Luke, he kind of hinted that, you know, I was asking actually, how do you compare Scrum teams and non-scrum teams? And he said, both have strengths, and one thing he mentioned is the downside of Scrum is too many meetings, especially the time it takes. Other teams don’t have it, so maybe they don’t want us to.
- (?):* Ah, so they don’t [inaudible]?
- Aamit:* No no, I’m just telling you what I heard.
- Mel:* So, ok, do you guys want it or you guys don’t want it? Liz [the SM] said it’s perfectly alright to do it without her.
- Max:* I’d like to have it, it has some good stuff, we can say what we did, we can improve...
- Joe:* Well that’s what it’s all about.  
[In the end, they decide to have the retrospective meeting, but limit it to 90 minutes in the afternoon.]

This excerpt shows how, in interaction with one another, engineers made sense of the current status of Scrum and engaged or disengaged with it accordingly. As the last section showed, engineers

already had a particular frame of interpretation for productivity schemes, but it was in these kinds of recurrent interactions that they solidified their shared view of Scrum and made decisions about how to act in relation to it. When Zian proposed skipping a meeting that Scrum required, he supported his proposal by giving an example of a team that had already decided to skip the same meeting. Another engineer underscored Zian's argument by observing that the second-level manager, Luke, did not seem to appreciate Scrum – a recurrent theme that would not have been salient if engineers believed Scrum would not be abandoned. In this case, the network team decided not to skip the meeting, but this interaction was one in a series of conversations in which engineers discussed whether and how to practice Scrum and its required strategies of empowerment.

Engineers, of course, had different opinions about Scrum. When I asked them about it individually, some of them replied favorably, stating that they like the teamwork and the opportunity to learn by taking on a new task or working with someone else. Others, on the other hand, said that Scrum did not work because it invited shirking and failed to account for individual engineers' extra ability or effort. No matter what each engineer's personal opinion of Scrum was, however, the engineers as a group used their collective understanding of Scrum's position in the organization to guide their actions and reactions in given situations. In these collective interpretations and reinterpretations of management actions, Scrum seemed to have fluctuating and unstable legitimacy.

The next excerpt is a negative case: it shows how an engineer who tried to act in accordance with Scrum had to backtrack when faced with a situation in which doing so would have negatively affected how managers perceived him. David, who had been appointed as a Scrum Master, had difficulty interacting with managers when they did not share the same opinions about Scrum:

When I visited one of the Scrum teams, I found that one engineer was taken off the Scrum Master role, which was subsequently given to David by the line manager. Talking to David, he complained that the decision to make him Scrum Master was a top-down decision. That was not in accordance of Scrum principles, he thought. Subsequently, this topic was discussed with Luke, the second level manager, when they had a communication meeting. The meeting goes on and the topic of Scrum Master rotation is discussed. Luke asked what was wrong with the line manager's decision.

*David:* This is the sphere of Scrum. I don't care if we do Scrum or not, I don't really care. But how big is the sphere? There is a huge gray zone. Then you are left going, can I do this, can I do that, and there is huge area of gray, and there is a tremendous amount of confusion.

*Laura:* You need a little more background, the reason we had this meeting was, one of the engineers was a Scrum Master and he said it takes too much time away from development. We wanted him to be a member. So we asked if someone from outside can be our Scrum Master to the line manager and we were told there was no one to do that, so we had to take responsibility on that as a team.

*Luke:* Did you have one?

*Laura:* We used to have Barney [who was reassigned to another team in the last reorganization].

*Luke:* We need to deal with the realities also, right?

*Laura:* I understand. I'm just saying how it came about as a background and so at that time, we were asked to decide if we wanted to rotate Scrum Master or not, and we really didn't have other options. I think that's why it got accepted.

*Luke:* Okay. The request was to have someone from outside. We didn't have that. And I agree that the team should have more comfortable discussion. Having said that, we

have to break this ‘I cannot tell him no in front of this person because he happens to be my manager’. I understand you are talking about fear, and,

*Sally:* I don’t think it’s only fear, I think part of it comes from culture too.

*David:* I too.

*Sally:* Yeah. Not everyone has the level of comfort in our team that other cultures have. So ... they keep quiet. Even though they strongly disagree. So among the peers they are more open.

[After the meeting, David went back to his office and started to compose an email to Bill, the manager who championed Scrum. When I asked him what he was doing, he answered, ‘I’m doing this to make sure that I am doing the right thing, and not get fired for making troubles. It’s funny because I’m just doing this [empowerment] because I’m the Scrum Master, that’s my job! But I am sure I am ‘the trouble guy’.

David’s action is strangely contradictory at first sight. He raised the issue of Scrum violation with his superiors: he told them that their decision to make him Scrum Master did not accord with Scrum’s principle of empowerment while at the same time explicitly stating that he cared about Scrum only to the extent that the managers wanted it. Luke, the second-level manager, responded that, according to Scrum principles, the team should be able to have comfortable discussions without second-guessing the manager’s intentions and holding back their own ideas. After the meeting, however, David did just the opposite of what Scrum principles would dictate: he reported his actions to another manager who he thought could validate what he had said as Scrum Master. In his interpretation, voicing his objections to a manager at this meeting made him ‘the trouble guy’.

David’s actions make sense when we look at them from the engineers’ perspective: they thought that they should engage with Scrum – because they had been instructed to – but they also thought that they should not be more committed to it than the managers were, since the managers might decide to abandon it in the future. Like the other engineers, David understood Scrum as potentially temporary and so tried to make sure that he was legitimately and properly empowered for both eventualities: Scrum’s continuation and its discontinuation. Mel, a mid-career engineer, succinctly summarized this perspective in an interview when I asked for his assessment of engineers’ commitment to Scrum:

I think people just wanted a clear direction. If it’s one way, let me know; I’ll work with it. If it’s the other, let me know; I’ll work with it. They want a direction, as opposed to someone saying some things and others saying other things. Of course, some people are more vocal than others, but I think in general people are fine as long as we know the directions. That’s the impression I got.

The engineers had created and maintained a shared understanding that all productivity schemes end sooner or later, and they scrutinized all management decisions as cues about Scrum’s legitimacy. As a result, their sense of Scrum’s legitimacy fluctuated, and they found it impossible to discern a single management ‘direction’. To put it differently, because the engineers could jointly interpret any and all management actions as early signs of potential Scrum abandonment, they were in a double bind of uncertainty. Precisely because they needed management’s constant reassurance of Scrum’s legitimacy, they could never actually get it. The engineers’ everyday interactions shaped and reshaped their understanding of Scrum in ways that guided their interpretation of work events. As a result, their conviction that Scrum would soon be undermined became a self-fulfilling prophecy: the engineers themselves undermined its legitimacy and refrained from committing to it, despite managers’ intentions to continue Scrum. The organization continued Scrum throughout my yearlong research period and for at least a year afterward, but Scrum never became

institutionalized: the engineers continued to assume that the scheme would eventually, and inevitably, end, and oriented their behavior accordingly.

## Discussion and Conclusion

This study used an interactionist ethnographic approach to understand how workers interpreted and responded to a productivity scheme through everyday interactions. The workers' employment of organizational memory created a culture that refracted management's intentions and shaped workers' actions in surprising ways. Regardless of the content of the productivity scheme and how well it was implemented, the workers engaged in 'tea-leaf reading' of managers' actions to discern whether and when the management would stop implementing the scheme. The workers created a group culture through which they constantly interpreted even small decisions or remarks by managers as potential signals of the scheme's fate and tried to engage with the scheme only to the extent that they believed that managers wanted them to at that given moment.

This study makes several theoretical and empirical contributions. First, it shows how organizational memory of past productivity schemes shaped the way engineers understood the implementation of Scrum. The shift from hierarchical top-down management to empowered teams was thorough: managers were no longer allowed to be part of production, and engineers were given more autonomy in software production. These organizational changes, however, were not enough to get software engineers to support Scrum and engage in empowered practices. Instead, the software engineers worked in a culture of uncertainty: they believed that Scrum would be short-lived and that they needed to constantly watch for signals from management that would imply its abandonment. Previous studies on the organizational memory, cultural schema, and organizational culture have not sufficiently shown how organizational memory and culture were linked and how the enactment of organizational memory shapes and reshapes culture. This study shows the process through which organizational memory influenced ongoing organizational life.

Organizational memory of abandoned 'processes' refracted the ways engineers understood the current situation. Organizational memory is both individual and social (Walsh & Ungson, 1991). The memory of past 'failed processes' was not just an individual memory; it was a shared narrative that guided engineers' collective efforts to make sense of their current situation. Engineers created and recreated their group culture through their collective interpretation of events. The interpretations and culture constituted each other; engineers' scripts guided their decisions about how to act, and their actions recreated their culture of uncertainty (Giddens, 1984).

Past studies on collective or organizational memory point out that memory is social: when people recall the past to make choices in the present, the memory is not a preset, objective set of knowledge about the past that they simply draw upon. Remembering is a creative process in which organizational members work to reconstitute certain aspects of the organization's past and, based on that reconstitution, figure out what they believe to be appropriate and legitimate actions in the current situation (Jansen, 2007; Wood, 2002). This work contributes to creating a collective perspective that workers use to navigate the changing organization. This study thus advances our understanding of how workers enact organizational memory – that is, how they put memory into action. The creation of workers' culture was essential in utilizing the organizational memory. The collective memory helped engineers devise their (re)action to the productivity scheme; but it was their shared discussions, interpretations, and stories that sustained their nonbelief and noncommitment. Without each other, engineers would not have had a feedback about whether their actions were still legitimate and appropriate. If they had been working in complete isolation, they may have had more disparate responses to the productivity scheme that would have depended on their individual experiences in the organization and their personal feelings about Scrum.

This study also illuminates how group culture is constructed and maintained in an organization, and how that refracts the management's intentions (Fine & Hallett, 2014). As I have emphasized, workers created and recreated their group culture in their interactions with one another. Instead of treating workers as victims of their own biases, the study highlighted workers as knowledgeable actors (Giddens, 1984) who have a good understanding of their structural position and their opportunities. Somewhat analogous to the story of working class kids' class reproduction by Willis (Willis, 1977; also see Giddens, 1984, pp. 88–99), it was precisely the engineers' initiatives that hollowed out a productivity scheme that was intended to benefit them. The engineers knew that Scrum was potentially temporary and that they had no control over its fate. One result was that, as they navigated mundane work situations, they subtly abandoned Scrum and its paradigm of empowerment more quickly than their managers did.

MIS studies have mentioned that a cultural shift to empowerment is an essential part of successful Scrum implementation, but these studies have not made clear just what such a cultural shift entails. In accord with this literature, I found that past work structure and practice seemed to have a persistent influence in the form of organizational memory and that this influence was independent of other aspects of the organization, including the new organizational structure, changed work practices, and managerial affirmation of a commitment to Scrum. What the study adds to our understanding of Scrum implementation is that the culture of engineers must be taken into account if an organization is to successfully implement Scrum. It is not enough to provide engineers with Scrum teams, resources, and even templates for practices; under conditions of change, engineers inevitably draw on their own interpretative schemes, expectations, and past experiences to create new practices or continue old ones, and these practices may quickly undermine the new work scheme.

The moral of this story for organizational practice is that when implementing a change, managers may be better off if they openly talk about the organization's history of failed productivity-scheme implementation, explain how they would like to implement the new productivity scheme differently, and invite workers' input in implementing the new scheme. By doing so, managers will gain some traction on how workers will employ organizational memory. Workers' collective memory is constrained by their earlier representations (Olick, 1999; Zelizer, 1995), so both refraining from talking about past schemes and embellishing past schemes' failures are likely to lead workers to simply reemploy the narrative of 'processes that fail and go'. But memory provides actors both with constraints and opportunities (Jansen, 2007), so if managers and workers acknowledge the failures of the past and commit to learning from them, then there is a possibility that workers won't create the self-fulfilling prophecy presented above.

The obvious challenge to implementing Scrum and other empowerment schemes is that managers are making a top-down decision to create a bottom-up working order. Empowerment demands autonomy among workers from managers. The ideal of flexibility means that workers must have control over their work and perhaps even more control in creating their own group culture. The philosophy of empowerment is steadily spreading in contemporary workplaces, but its implications for workers' everyday experiences have been understudied. When we study workers' everyday experiences with empowerment schemes, it becomes clear why they might not experience management-directed empowerment as empowering. It's possible that, at least in some contexts, 'empowerment' generates more anxiety and frustration than the 'old and rigid' hierarchical structure did. It is only by taking workers' perspectives into account that we can accurately depict the ways that workplaces and organizations are changing and the ways that these changes affect workers' everyday lives.

## Acknowledgements

I would like to thank Nina Eliasoph, Jack Katz, Gabriel Rossman, Iddo Tavory, Stefan Timmermans, Jack and Marilyn Whalen, Lynne G. Zucker, and *Organization Studies* editors and reviewers for their careful comments and criticisms.

## Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

## References

- Abrahamson, E., & Fairchild, G. (1999). Management fashion: Lifecycles, triggers, and collective learning processes. *Administrative Science Quarterly*, 44, 708–740.
- Abrahamsson, P., Warsta, J., Siponen, M. T., & Ronkainen, J. (2003). New directions on agile methods: A comparative analysis. *Proceedings of the 25th International Conference on Software Engineering*, 244–254.
- Anteby, M., & Molnar, V. (2016). Collective memory meets organizational identity: Remembering to forget in a firm's rhetorical history. *Academy of Management Journal*, 55(3), 515–540.
- Barley, S. R., & Tolbert, P. S. (1997). Institutionalization and structuration: Studying the links between action and institution. *Organization Studies*, 1, 93–117.
- Bechky, B. A. (2011). Making organizational theory work: Institutions, occupations, and negotiated orders. *Organization Science*, 22(5), 1157–1167.
- Becker, H. S., Geer, B., Hughes, E. C., & Strauss, A. L. (1977). *Boys in white: Student culture in medical school*. New York: Transaction.
- Beer, M., Eisenstat, R., & Spector, B. (1990). Why change programs don't produce change. *Harvard Business Review*, 67(6), 158–166.
- Boehm, B., & Turner, R. (2005). Management challenges to implementing agile processes in traditional development organizations. *IEEE Software*. September/October: 30–39.
- Cohn, M. (2010). *Succeeding with Agile: Software development using Scrum*. Boston: Pearson Education.
- Emerson, R. M., Fretz, R. I., & Shaw, L. L. (1995). *Writing ethnographic fieldnotes*. Chicago: University of Chicago Press.
- Emirbayer, M., & Mische, A. (1998). What is agency? *American Journal of Sociology*, 103, 962–1023.
- Fayard, A., & Van Maanen, J. (2015). Making culture visible: Reflections on corporate ethnography. *Journal of Organizational Ethnography*, 4(1), 4–27.
- Fiedler, M., & Welp, I. (2010). How do organizations remember? The influence of organizational structure on organizational memory. *Organization Studies*, 31, 381–407.
- Fine, G. A. (1979). Small groups and culture creation: The idioculture of little league baseball team. *American Sociological Review*, 44, 773–745.
- Fine, G. A. (1984). Negotiated orders and organization cultures. *Annual Review of Sociology*, 10, 239–262.
- Fine, G. A., & Hallett, T. (2014). Group cultures and the everyday life of organizations: Interaction orders and meso-analysis. *Organization Studies*, 35(12), 1773–1792.
- Ford, J. D., Ford, L. W., & D'Amelio, A. (2008). Resistance to change: The rest of the story. *Academy of Management Review*, 33(2), 362–377.
- Giddens, A. (1984). *The constitution of society: Outline of the theory of structuration*. Cambridge, MA: Polity Press.
- Glaser, B. G., & Strauss, A. L. (1967). *The discovery of grounded theory*. New York: Aldine.
- Halbwachs, M. (1980). *The collective memory*. New York: Harper Colophon Books.
- Hallett, T. (2010). The myth incarnate: Recoupling processes, turmoil, and inhabited institutions in an urban elementary school. *American Sociological Review*, 75, 52–74.
- Hallett, T., & Ventresca, M. (2006). Inhabited institutions: Social interactions and organizational forms in Gouldner's patterns of industrial bureaucracy. *Theory and Society*, 35, 213–236.
- Hannan, M. T., & Freeman, J. (1984). Structural inertia and organizational change. *American Sociological Review*, 49(2), 149–164.
- Jansen, J. S. (2007). Resurrection and appropriation: Reputational trajectories, memory work, and the political use of historical figures. *American Journal of Sociology*, 112, 953–1007.
- Kanter, R. M. (1989). *When giants learn to dance: Mastering the challenges of strategy, management and careers in the 1990s*. New York: Simon and Schuster.

- Kanter, R. M., Stein, B. A., & Jick, T. D. (1992). *The challenge of organizational change*. New York: The Free Press.
- Katz, J. (2001). Analytical induction. In N. J. Smelser & P. B. Baltes (Eds.), *International encyclopedia of the social and behavioral sciences* (pp. 480–484). Oxford: Elsevier.
- Kirkman, B. L., & Rosen, B. (1999). Beyond self-management: Antecedents and consequences of team empowerment. *Academy of Management Journal*, 42, 58–74.
- Kirkman, B. L., & Shapiro, D. L. (1997). The impact of cultural values on employee resistance to teams: Toward a model of globalized self-managing work team effectiveness. *The Academy of Management Review*, 22, 730–757.
- Labianca, G., Gray, B., & Brass, D. J. (2000). A grounded model of organizational schema change during empowerment. *Organization Science*, 11, 235–257.
- Larman, C. (2004). *Agile & iterative development: A management's guide*. New York: Pearson Education.
- Leach, L. P. (2000). *Critical chain project management*. Boston, MA: Artech House.
- Moe, N. B., Dingsøyr, T., & Dybå, T. (2010). A teamwork model for understanding an Agile team: A case study of a Scrum project. *Information and Software Technology*, 52, 480–491.
- Moorman, C., & Miner, A. S. (1998). Organizational improvisation and organizational memory. *Academy of Management Review*, 23, 698–723.
- Morrill, C. (2008). Culture and organization theory. *Annals of the American Academy of Political and Social Science*, 619, 15–40.
- Mumby, D. K. (2005). Theorizing resistance in organization studies: A dialectical approach. *Management Communication Quarterly*, 19(1), 1–26.
- Nerur, A., Mahapatra, R., & Mangalaraj, G. (2005). *Communications of the ACM*, 48(5), 72–79.
- Olick, J. K. (1999). Collective memory: The two cultures. *Sociological Theory*, 17, 333–348.
- Olick, J. K. (2003). *States of memory: Continuities, conflicts, and transformations in national retrospection*. Durham: Duke University Press.
- Olick, J. K. (2007). *The politics of regret: On collective memory and historical responsibility*. London: Routledge.
- Olick, J. K., & Robbins, J. (1998). Social memory studies: From ‘collective memory’ to the historical sociology of mnemonic practices. *Annual Review of Sociology*, 24, 105–140.
- Orr, J. E. (1996). *Talking about machines: An ethnography of a modern job*. Ithaca, NY: Cornell University Press.
- Ouchi, W. G. (1980). Markets, bureaucracies, and clans. *Administrative Science Quarterly*, 25, 129–141.
- Ouchi, W. G. (1981). *Theory Z*. Reading, MA: Addison-Wesley.
- Pfeffer, J. (1981). Management as symbolic action: The creation and maintenance of organizational paradigms. *Research in Organizational Behavior*, 3, 1–52.
- Palmer, S. R., & Felsing, J. M. (2002). *A practical guide to feature-driven development*. Upper Saddle River, NJ: Prentice Hall.
- Paulk, M. C., Weber, C. V., Curtis, B., & Chrissis, M. B. (1995). *The capability maturity model: Guidelines for improving the software process*. Indianapolis, IN: Addison-Wesley Professional.
- Rowlinson, M., Booth, C., Clark, P., Delahaye, A., & Procter, S. (2009). Social remembering and organizational memory. *Organization Studies*, 31, 69–87.
- Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26, 1–9.
- Schaffer, R. H., & Thomson, H. A. (1992). Successful change programs begins with results. *Harvard Business Review*, January–February, 80–89.
- Schein, E. H. (1985). *Organizational culture and leadership: A dynamic view*. San Francisco, CA: Jossey-Bass.
- Schwaber, K. (2004). *Agile project management with Scrum*. Redmond, WA: Microsoft Press.
- Sheridan, J. E. (1992). Organizational culture and employee retention. *Academy of Management Journal*, 35(5), 1036–1056.
- Swidler, A. (1986). Culture in action: Symbols and strategies. *American Sociological Review*, 51, 273–286.

- Tavory, I., & Timmermans, S. (2014). *Abductive analysis: Theorizing qualitative research*. Chicago: University of Chicago Press.
- Wadhvani, R. D., & Bucheli, M. (2014). The future of the past in management and organization studies. In R. D. Wadhvani & M. Bucheli (Eds.), *Organizations in time: History, theory, methods*. New York: Oxford University Press.
- Walsh, J. P., & Ungson, G. R. (1991). Organizational memory. *The Academy of Management Review*, 16, 57–91.
- Weeks, J. (2003). *Unpopular culture: The ritual of complaint in a British bank*. Chicago: University of Chicago Press.
- Willis, P. (1977). *Learning to labor: How working class kids get working class jobs*. New York: Columbia University Press.
- Wood, M. (2002). Mind the gap? A processual reconsideration of organizational knowledge. *Organization*, 9(1), 151–171.
- Zelizer, B. (1995). Reading the past against the grain: The shape of memory studies. *Critical Studies in Mass Communication*, 12, 214–39.
- Zucker, L. G. (1977). The role of institutionalization in cultural persistence. *American Sociological Review*, 42, 726–743.

### Author biography

Nahoko Kameo is an Assistant Professor/Faculty Fellow of Sociology at New York University. She holds a PhD from UCLA. Her intellectual interest lies in studying innovation and entrepreneurship from the lenses of organizational theory. Her most recent project seeks to understand the transformation of entrepreneurial practices among Japanese university scientists under new policy initiatives. Her recent work has been published in *Theory and Society* and *Qualitative Sociology*.